Guía para defender y explicar el sitio

Proyecto Gestión de Clientes · nivel inicial · preguntas y respuestas

Objetivo: que puedas contar qué problema resuelve el sitio, cómo funciona cada parte y qué ocurre con los datos, además de responder preguntas típicas sobre la implementación.

Importante: las respuestas entre comillas son modelos para practicar. Cuéntalas con tus palabras y no presentes como terminado algo que el código aún no guarda o implementa.

1. Presentación del proyecto

Tema	Explicación / respuesta sugerida
Qué es	Es un CRM web para organizar clientes por zona y consultar información comercial: última compra, frecuencia estimada de reposición, pedido y deuda.
Cómo explicarlo en 30 segundos	«La interfaz está hecha con React. Cuando necesita guardar o consultar clientes, se comunica por HTTP con una API de Express. El servidor centraliza las operaciones y usa Sequelize para trabajar con MySQL. Así se separan la pantalla, la lógica del servidor y los datos.»
Qué necesidad resuelve	Reúne la información de los clientes, permite encontrarla por zona y tener a la vista quién tiene deuda o podría necesitar reposición.
A quién está dirigido	A una persona que gestiona clientes y ventas de mercadería. La interfaz incluye nombre/contacto, ubicación, producto, montos, frecuencia y un enlace de WhatsApp.
Qué significa full-stack	Que el proyecto incluye tanto el frontend (lo que usa la persona) como el backend (API y acceso a datos), además de una base de datos.
Tecnologías	Frontend: React, React Router, Redux/Redux Thunk y Axios. Backend: Node.js, Express, Sequelize y MySQL. CSS define la apariencia.
Límite de la explicación	Conviene describir lo que realmente hace el código. Algunas funcionalidades anunciadas en el diseño aún son parciales o no se guardan en la base de datos; se detallan más adelante.

2. Recorrido de la aplicación y pantallas

Tema	Explicación / respuesta sugerida
Inicio	La pantalla Landing ofrece accesos a Interior, Corrientes y Chaco. React Router cambia de pantalla dentro de la aplicación sin recargar toda la página.
Pantalla de zona	La vista pide la lista de clientes a la API, filtra por provincia y presenta un buscador. Luego separa los resultados en «Con deuda pendiente» y «Clientes al día».
Crear cliente	Desde la pantalla de zona se abre un formulario. Al enviarlo, se valida la provincia, se calcula total/deuda, se intenta evitar un duplicado y se manda una petición al backend.
Tarjeta de cliente	Cada tarjeta presenta información y acciones para abrir WhatsApp, editar, saldar deuda o eliminar. La pantalla de zona le pasa al componente las funciones necesarias.
Chaco y Corrientes	Son vistas muy parecidas: cargan los datos, filtran su provincia y muestran tarjetas. Cambia principalmente la provincia y los mensajes.
Interior	Hace el mismo recorrido, pero utiliza una función reutilizable para dibujar las secciones de deuda y al día.
Buscador	Filtra en el navegador los clientes cargados usando nombre, contacto o localidad; no hace una nueva consulta al servidor por cada letra.
Archivos de entrada	my-app/src/index.js monta React y los proveedores de Redux y rutas. App.js asocia URLs a pantallas. views/index.js reúne las exportaciones de vistas.

3. Del navegador a la base de datos

Tema	Explicación / respuesta sugerida
1. Interfaz	Una vista o formulario captura una acción, por ejemplo enviar un formulario para crear un cliente.
2. Petición HTTP	Axios o fetch manda GET, POST, PUT o DELETE a una dirección como http://localhost:3001/clientes. HTTP es el protocolo de comunicación.
3. Express y rutas	Server/src/app.js configura Express. Las rutas conectan una dirección y un método HTTP con la función que debe atenderlos.
4. Controlador	El controlador recibe los datos, aplica la lógica de esa operación y decide qué respuesta HTTP devolver.
5. Sequelize y modelo	El controlador usa el modelo de Cliente. Sequelize traduce las operaciones de JavaScript a consultas SQL.
6. MySQL	La base almacena o devuelve registros. El resultado vuelve por la ruta hasta el navegador, que actualiza lo que muestra.
CORS	Es una regla del navegador sobre qué origen puede llamar a la API. El servidor permite el frontend local en el puerto 3000; CORS no es un inicio de sesión ni reemplaza autenticación.
Estado local y Redux	React useState guarda valores de una pantalla o tarjeta. Redux mantiene un estado compartido, aunque en este proyecto las vistas de zona también conservan sus listas localmente.

4. Qué hace cada parte del código

Tema	Explicación / respuesta sugerida
React y JSX	React organiza la interfaz en componentes. JSX es la sintaxis parecida a HTML que aparece dentro de JavaScript para describir lo que se dibuja.
Props	Son datos o funciones que un componente recibe de otro. Por ejemplo, la tarjeta recibe el cliente y las acciones para editarlo o borrarlo.
useState	Hook de React que recuerda un valor y vuelve a renderizar el componente cuando ese valor cambia.
useEffect	Hook usado para ejecutar efectos, como cargar datos al entrar en una vista o cuando cambia una dependencia.
Map y filter	map transforma cada elemento en una tarjeta; filter crea una lista con solo los elementos que cumplen una condición.
Axios / fetch	Herramientas para realizar solicitudes HTTP desde el navegador. Axios se usa en las acciones Redux y vistas; el formulario también usa fetch.
Redux actions	Describen o realizan una operación: leer, crear, actualizar o eliminar. Redux Thunk permite que una acción espere una llamada HTTP antes de despachar el resultado.
Reducer y store	El reducer calcula el nuevo estado a partir del estado anterior y una acción. El store es el lugar global donde Redux guarda ese estado.
Express middleware	Funciones que se ejecutan durante una petición. Aquí JSON interpreta el cuerpo, Morgan registra solicitudes, CORS aplica reglas de origen y otro middleware devuelve errores.
Modelo / controlador / ruta	El modelo define los datos; el controlador implementa la operación; la ruta decide qué controlador atiende cada URL.

5. Reglas de negocio que debes poder explicar

Tema	Explicación / respuesta sugerida
Próxima compra	La tarjeta suma la frecuencia en días a la última compra. Si falta frecuencia utiliza 7 días. No predice una venta real: es una estimación según los datos ingresados.
Semáforo	Sin fecha: gris. Fecha pasada: rojo. Hoy o dentro de los próximos dos días: amarillo. Más adelante: verde.
Total del pedido	Cuando hay cantidad y precio por kilo, el total es cantidad × precio por kilo.
Deuda	El formulario calcula máximo(0, total − pagado), por lo que no muestra deuda negativa si se pagó de más.
Deuda en tarjeta	La tarjeta usa la deuda guardada si está disponible; para mostrar el total puede recurrir a monto pagado + deuda cuando falta cantidad/precio.
Clientes al día	La vista incluye clientes cuya deuda convertida a número es exactamente cero. La sección con deuda incluye valores mayores que cero.
Saldar	La acción de la tarjeta pide confirmación y manda al servidor una actualización con deuda en cero y monto igual al total calculado.
WhatsApp	El botón abre una URL externa wa.me con teléfono y mensaje prellenado. No es una API de WhatsApp ni envía el mensaje por sí solo.
Formato monetario	toLocaleString('es-AR') da formato local a los importes y fechas. Los cálculos se hacen con números; el texto formateado es para mostrar.

Ejemplo: 100 kg × $500/kg = $50.000 de total. Si se pagaron $20.000, deuda = máximo(0, $50.000 − $20.000) = $30.000.

6. API: qué puede hacer el servidor

Tema	Explicación / respuesta sugerida
GET /clientes	Lista todos los clientes. Si recibe el parámetro name, filtra por nombre sin distinguir mayúsculas.
GET /clientes/:id	Busca un cliente por su identificador.
POST /clientes	Crea un cliente con los campos que el controlador entrega al modelo. Si no llega deuda, le asigna cero.
PUT /clientes/:id	Busca el cliente, responde 404 si no existe, actualiza los campos admitidos y guarda los cambios.
DELETE /clientes/:id	Borra el registro que coincide con el id.
GET /pedidos	Existe una ruta de prueba que devuelve un texto. No es todavía una API completa de administración de pedidos.
Código de estado	200 indica respuesta exitosa en varias rutas; 404 indica que no se encontró el cliente al actualizar; 500 indica un error interno.
Arranque	Server/src/index.js verifica la conexión a MySQL y después escucha en el puerto 3001. El frontend se inicia aparte en el 3000.

7. Preguntas que podrían hacerte (respuestas para practicar)

Tema	Explicación / respuesta sugerida
¿Por qué separaron frontend y backend?	«Para separar responsabilidades: React se ocupa de la experiencia visual; Express expone las operaciones y protege la lógica de acceso a datos; MySQL conserva la información.»
¿Qué pasa al buscar un cliente?	«La pantalla ya tiene la lista de su zona. Filtra en memoria por nombre, contacto y localidad y React actualiza la lista visible mientras escribo.»
¿Cómo sabe a qué zona pertenece?	«La ruta renderiza la vista con una provincia y esa vista conserva solo los clientes cuyo campo provincia coincide, sin importar mayúsculas.»
¿Qué ocurre al guardar un cliente?	«El formulario valida la zona, consulta clientes para detectar duplicados, calcula total/deuda y despacha la acción de creación. La acción hace POST a la API y el controlador crea el registro usando Sequelize.»
¿Cómo se calcula la deuda?	«Total del pedido menos lo pagado, con un mínimo de cero. El total se calcula con kilos por precio por kilo.»
¿Qué hace Redux aquí?	«Comparte el estado entre componentes y recibe acciones de lectura o cambios. Thunk permite esperar llamadas al backend antes de despachar el resultado.»
¿Qué diferencia hay entre estado local y global?	«El estado local pertenece a una vista o tarjeta, como el texto del buscador o el modo edición. El global de Redux se comparte con toda la aplicación.»
¿Qué es Sequelize?	«Es un ORM: permite definir modelos y consultar MySQL desde JavaScript sin escribir cada consulta SQL a mano.»
¿Qué es una API REST?	«Es una forma de exponer recursos con URLs y métodos HTTP. Aquí clientes se consulta con GET, crea con POST, modifica con PUT y elimina con DELETE.»
¿WhatsApp envía el mensaje automáticamente?	«No. Abre WhatsApp con un mensaje escrito de antemano; la persona todavía tiene que revisar y enviarlo.»
¿Hay una tabla/funcionalidad de pedidos completa?	«Se definió un modelo básico y una relación cliente-pedidos, pero la ruta de pedidos actual es solo de prueba. La interfaz muestra datos de último pedido en la tarjeta, pero la API de pedidos todavía no tiene operaciones CRUD completas.»
¿Qué validarías antes de publicar?	«Que todas las propiedades se guarden, que la API maneje errores y validaciones, que haya pruebas reales, que las credenciales sean seguras y que exista autenticación si los datos son privados.»

Tip: responde primero en una frase sencilla y después da un ejemplo del proyecto. Así demuestras comprensión sin perderte en detalles.

8. Limitaciones reales que conviene conocer

Tema	Explicación / respuesta sugerida
Campos que no persisten	El formulario y las tarjetas manejan contacto, frecuencia, producto, marca, cantidadKg y precioKg, pero el modelo y los controladores de clientes no definen ni guardan esos campos. La interfaz puede mostrarlos temporalmente, pero MySQL no los conserva.
Respuesta de creación y Redux	POST responde con un objeto que contiene message y cliente. La acción despacha la respuesta completa como POST_CLIENTE y el reducer la agrega como si fuera un cliente; eso no tiene la misma forma que un registro de cliente.
Duplicados	La comprobación del formulario compara nombre + provincia, pero el modelo declara name como único globalmente. Así, el mismo nombre en dos provincias puede pasar la comprobación del frontend y luego ser rechazado por MySQL.
Búsqueda Redux	La pantalla usa búsqueda local. Además, la acción searchByName no coincide con el parámetro que espera la ruta y el reducer espera un texto donde la acción parece enviar una lista. No conviene afirmar que esa búsqueda Redux está funcionando.
Sin login	En el código revisado no hay autenticación ni autorización de usuarios. CORS por sí solo no impide que un cliente no autorizado llame a una API accesible.
Configuración local	Las direcciones HTTP apuntan a localhost; para producción deberían configurarse por entorno. La configuración de MySQL también debe usar variables de entorno y nunca exponerse en un repositorio.
Manejo de errores	Algunas acciones registran errores en consola, pero las pantallas pueden actualizarse localmente antes de confirmar que la API completó el cambio; si falla, pantalla y servidor podrían diferir.
Pruebas	El test visible es el ejemplo inicial de Create React App («learn react»), no comprueba el flujo del CRM. Harían falta pruebas propias de formulario, reglas y endpoints.
Validación	Hay validación de provincia y chequeo de duplicado en el formulario, además de restricciones del modelo. No es una estrategia completa: el servidor debería validar los datos de forma consistente.

9. Cómo mostrarlo y cómo responder con seguridad

Tema	Explicación / respuesta sugerida
Demostración en 5 pasos	1) Abrir inicio y elegir una zona. 2) Explicar buscador y grupos de deuda. 3) Abrir una tarjeta y explicar semáforo/importe. 4) Mostrar el formulario y el cálculo automático. 5) Explicar que guardar llama a la API y que el servidor consulta MySQL.
Si no sabes algo	«No quiero afirmarlo sin comprobarlo. En el código que revisé, esta parte está implementada de esta manera…» Es mejor delimitar lo que sabes que inventar un comportamiento.
Si preguntan por una mejora	Una respuesta concreta puede ser: alinear modelo/controlador con los campos del formulario, agregar validación backend y pruebas, y mover configuración sensible a variables de entorno.
Si preguntan qué aprendiste	«Aprendí cómo se conecta una interfaz React con una API, cómo viaja un dato hasta MySQL y cómo separar componentes, estado, rutas y modelos.»
Para ejecutar localmente	Hace falta Node.js y MySQL configurado. Asegura la base que indica Server/src/db.js; inicia el servidor desde Server con npm install (la primera vez) y npm start; en otra terminal inicia my-app del mismo modo.
Consejo para estudiar	Primero memoriza el recorrido frontend → API → controlador → modelo → MySQL y el camino inverso. Después practica explicar un cliente: cómo se crea, se muestra, se filtra, se edita y se elimina.